

CS & IT ENGINEERING

DATA STRUCTURE

AVL Tree



JOIN TELEGRAM TO ENROLL

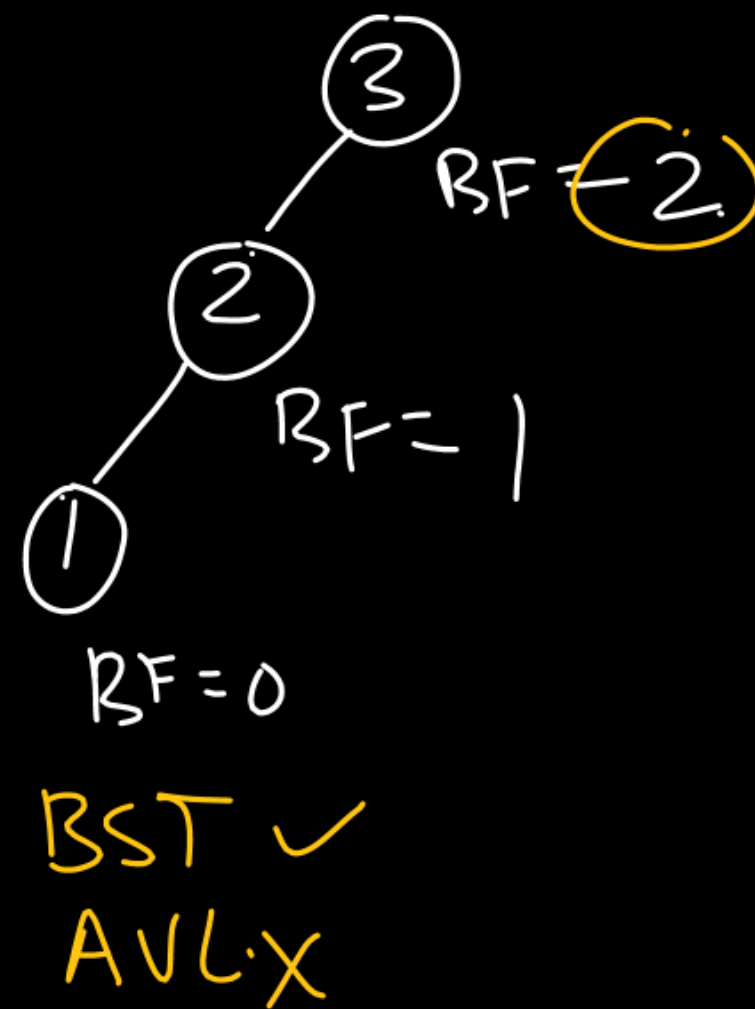
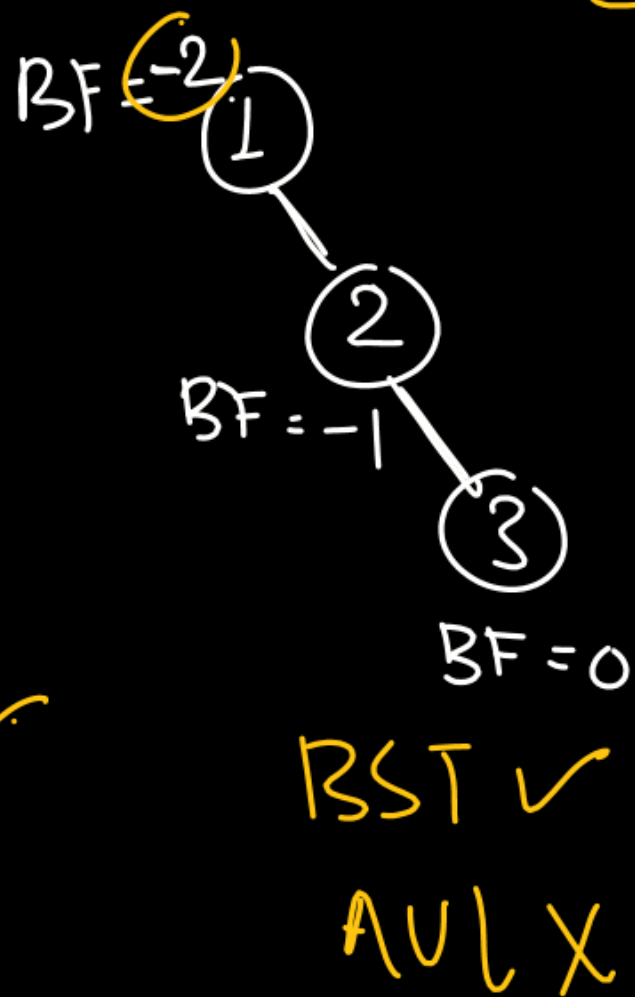
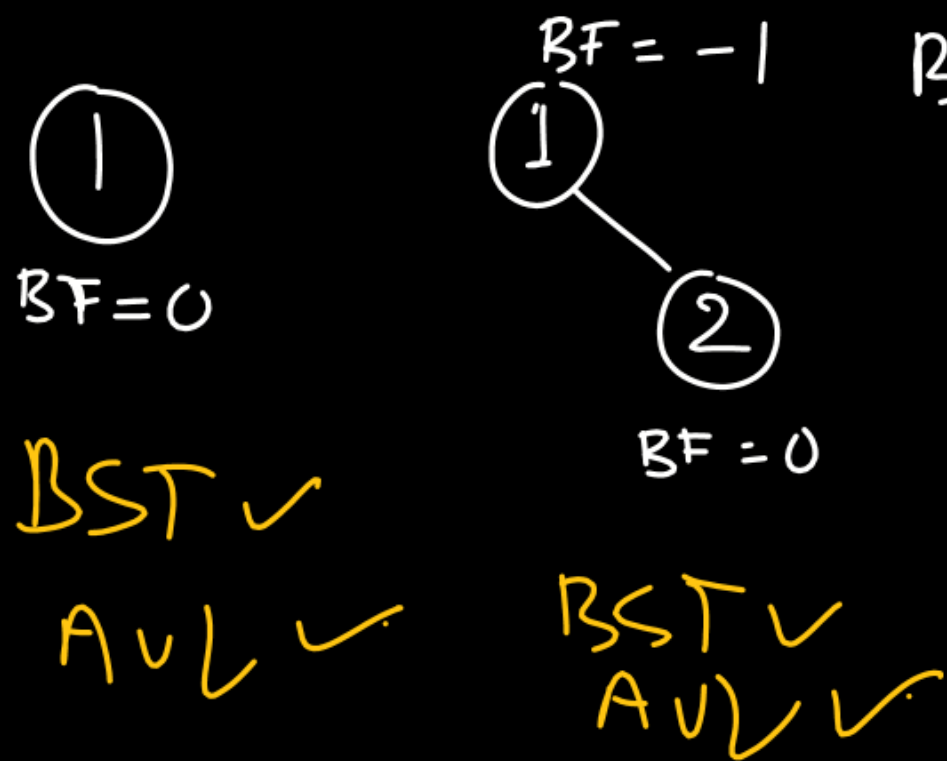
SHAIENDRA SIR

AVL Tree (Height Balanced Tree)

- ✓• AVL Tree
- ✓• Rotation in AVL Tree
- ✓• Construction of AVL Tree
- ✓• Operation on AVL Tree
- ✓• Properties of AVL Tree

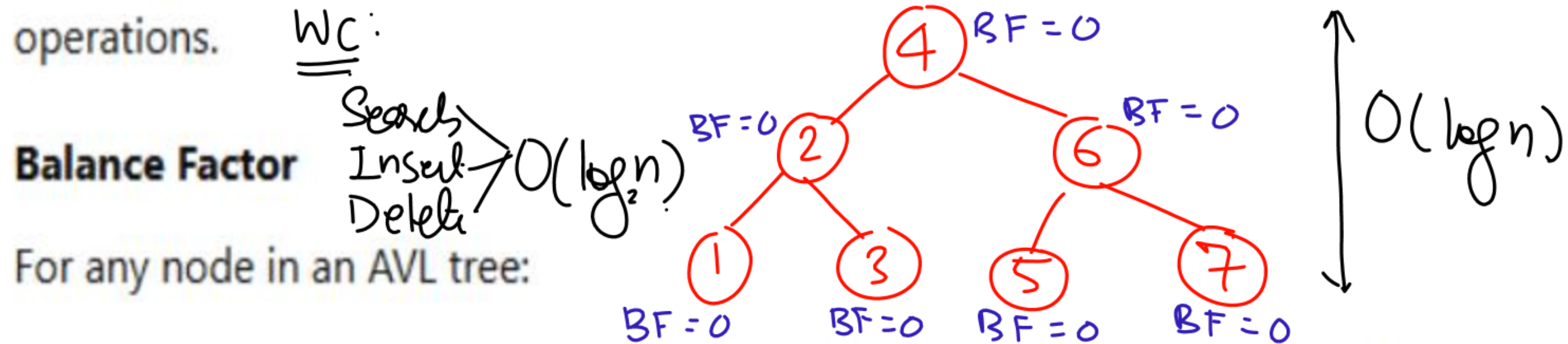
AVL Tree :- $BF = -1, 0, 1$

$$\underline{BF} = H(LST) - H(RST) = \underline{-1, 0, 1}$$



AVL Tree (Height Balanced Tree)

An AVL Tree is a type of self-balancing binary search tree (BST) where the difference between the heights of the left and right subtrees of any node (called the **balance factor**) is at most 1. This ensures that the tree remains balanced, leading to $O(\log n)$ time complexity for insertion, deletion, and search operations.



Balance Factor = Height of Left Subtree – Height of Right Subtree

A node is considered **balanced** if the balance factor is -1, 0, or 1. If it goes beyond this range, rotations are performed to restore balance.

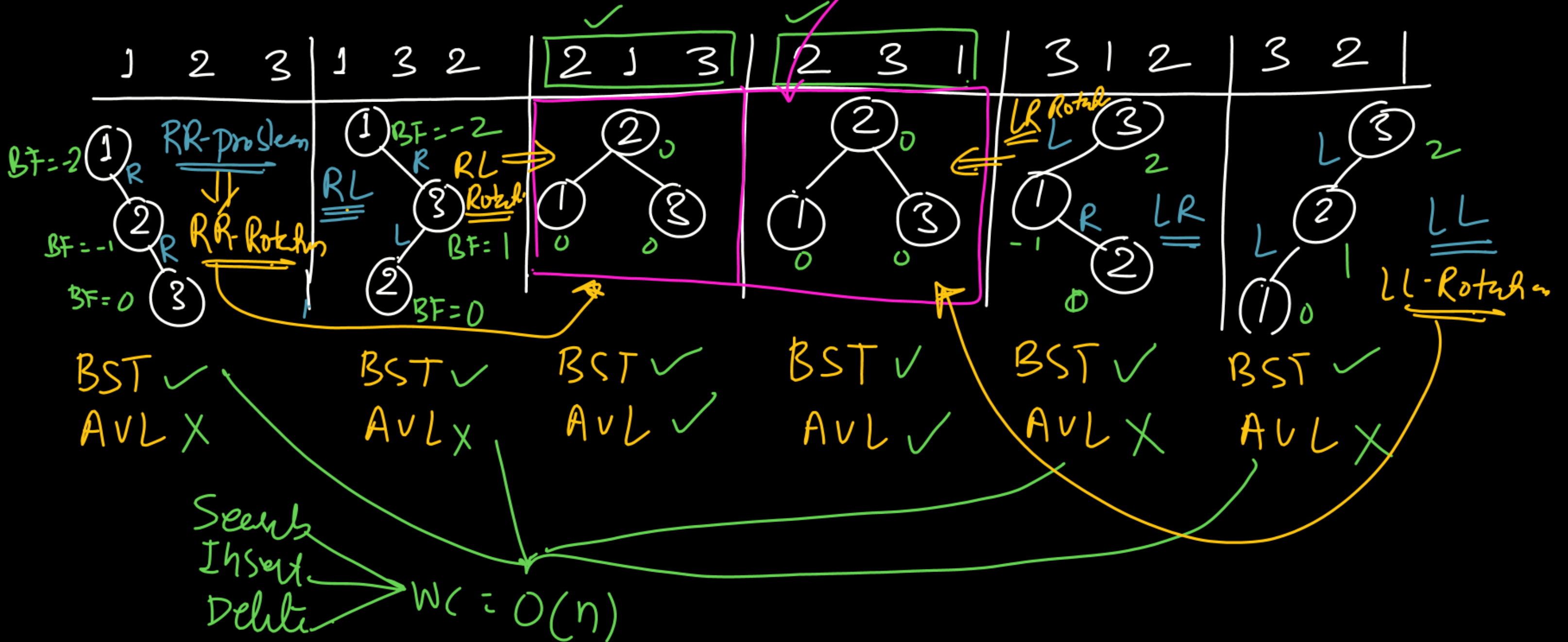
Why AVL Tree ?

if $n=3$ (1, 2, 3)

left < Root < Right

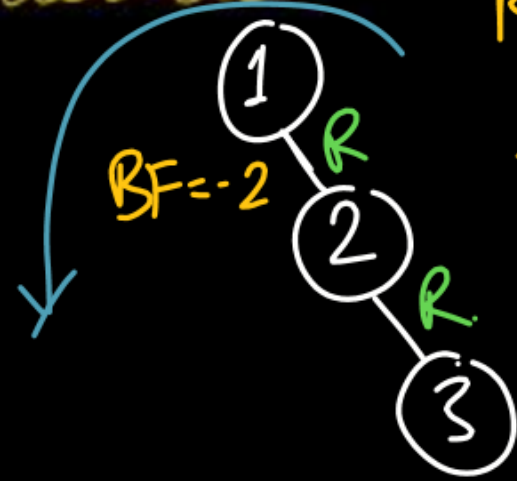
BF = -1, 0, 1

$WC = O(\log n)$ ← Search
Insert
Delete



Rotations in AVL Tree

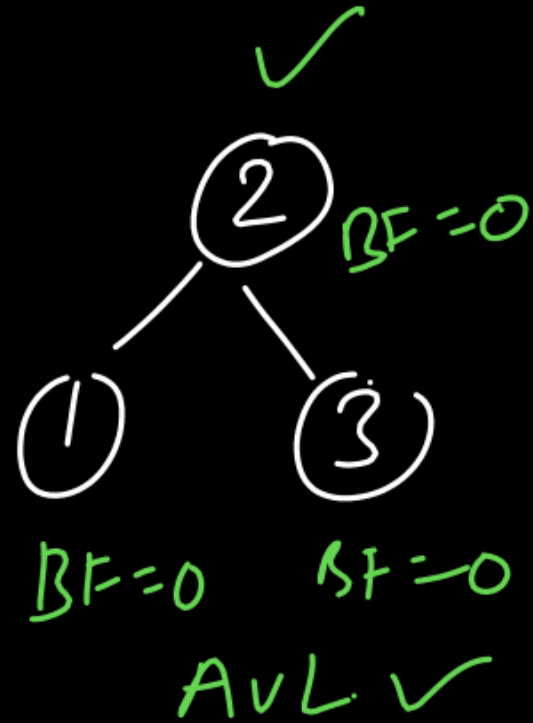
RR-Rotation:-



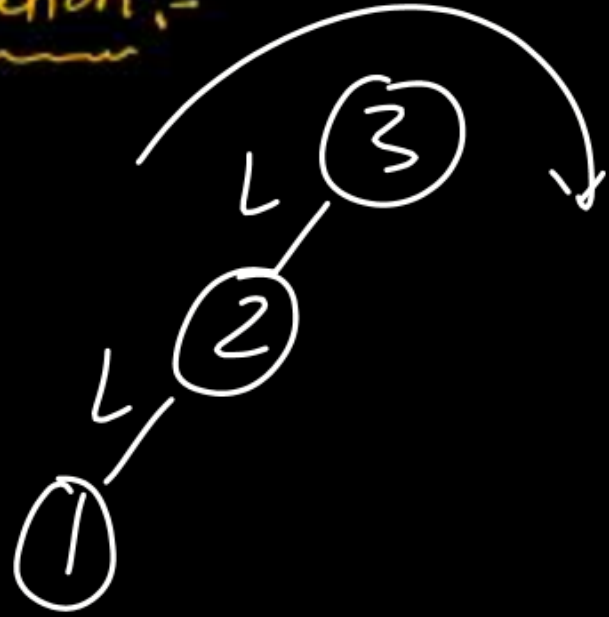
RR-Rotation.

Left Rotate

(Single Rotation)



LL-Rotation:-



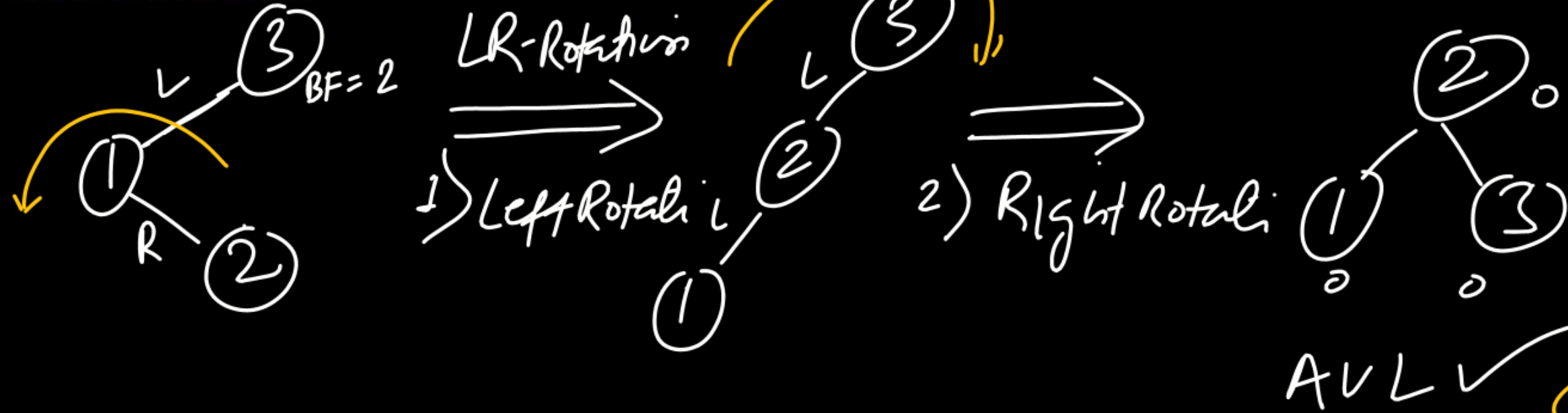
LL-Rotation

Right Rotate

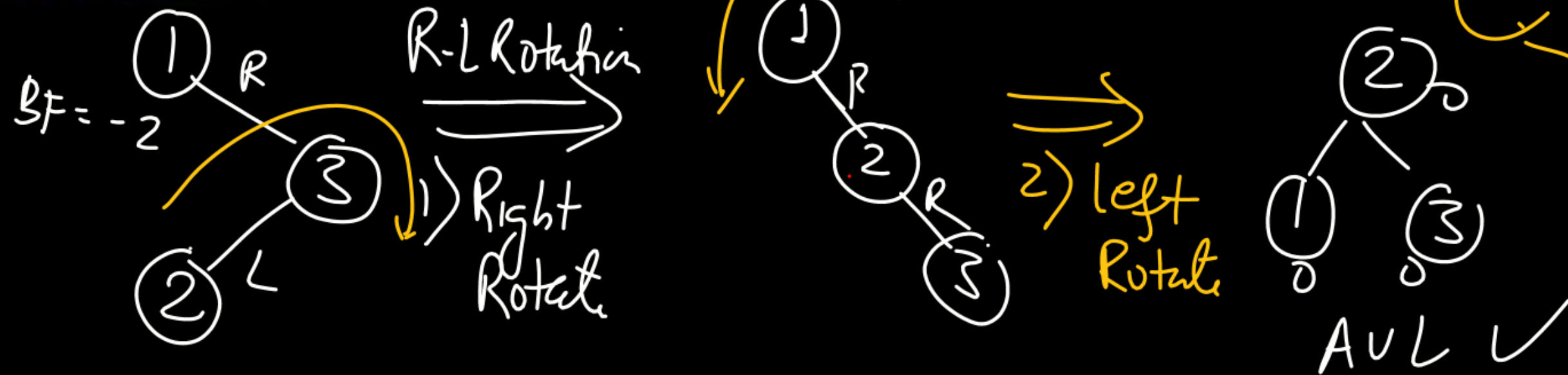
(Single Rotation)



LR-Rotation :- 2 Rotations



RL-Rotation :-



Rotation

$$T.C = O(1)$$

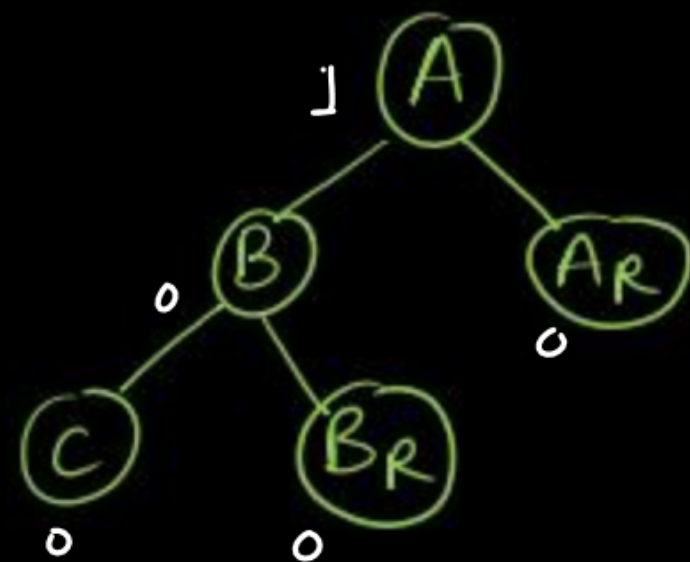
NOTE: BST Can be converted to AVL-Tree in $O(1)$ time Simply done 1 or 2 Rotation

How to balance node:-

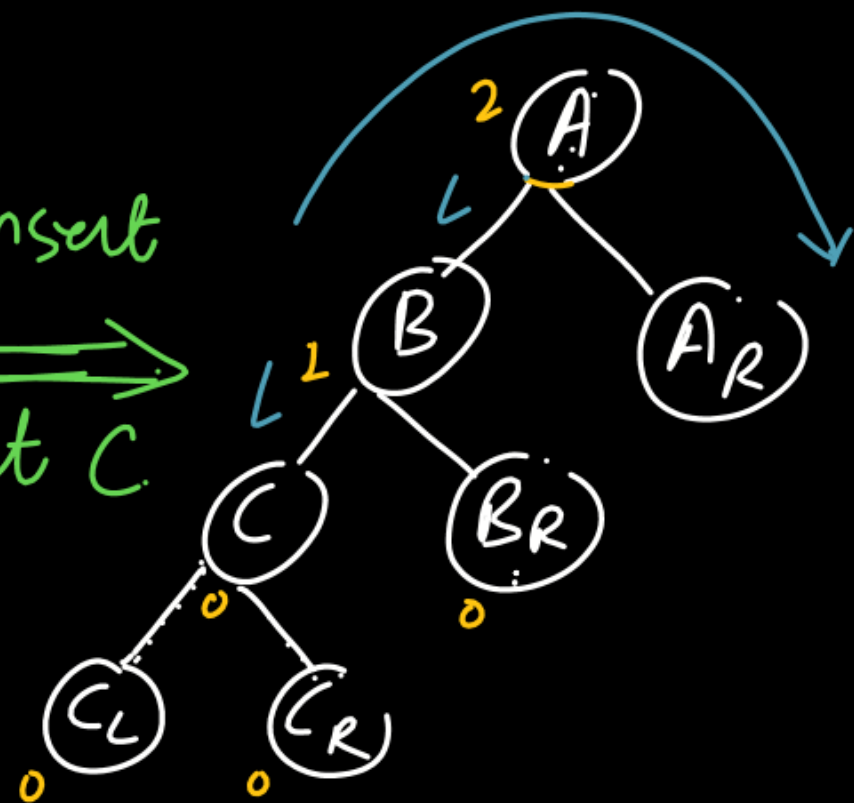
- Count three nodes in the direction of Leaf node
- Then, use the concept of AVL tree rotations to rebalance the tree

- ✓ LL Case → Right Rotation (Single Rotation)
- ✓ RR Case → Left Rotation (Single Rotation)
- ✓ LR Case → Left Rotation, then Right Rotation (Double Rotation)
- ✓ RL Case → Right Rotation, then Left Rotation (Double Rotation)

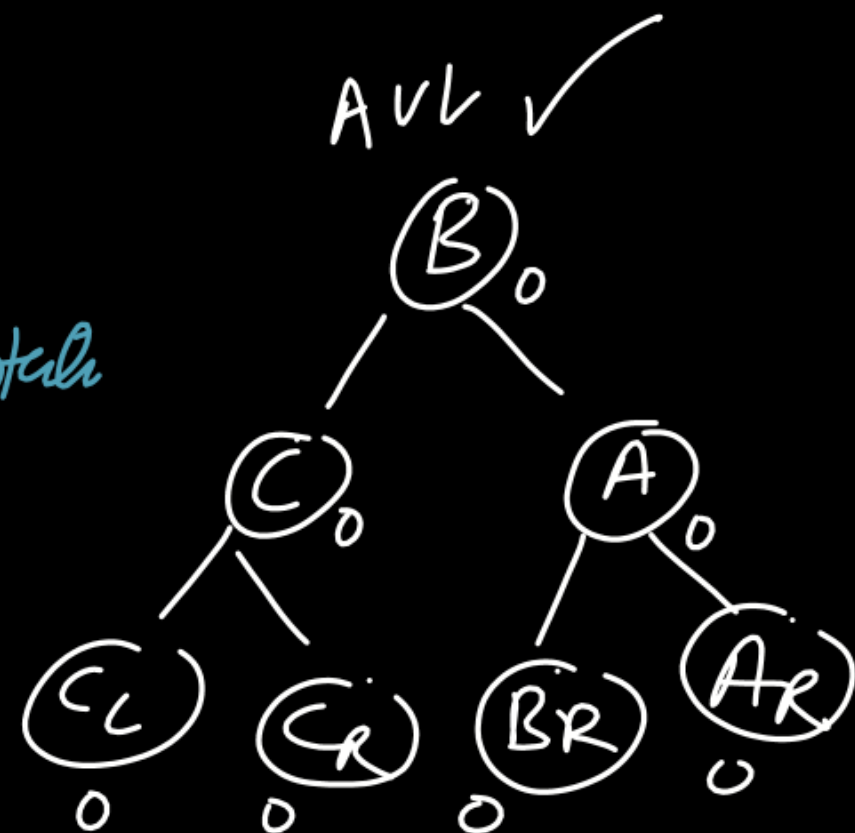
LL-Problem:-



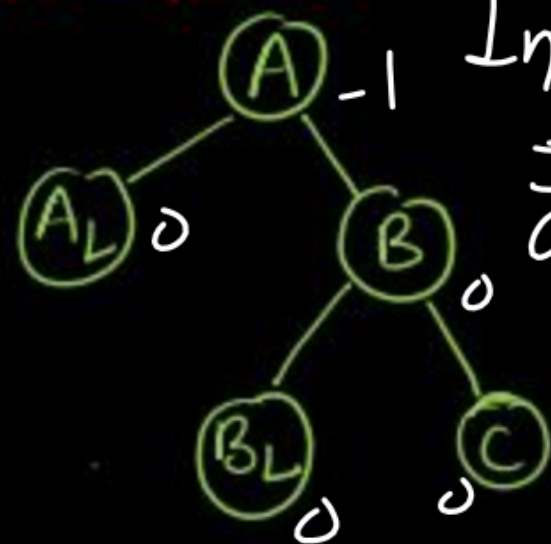
Insert
at C



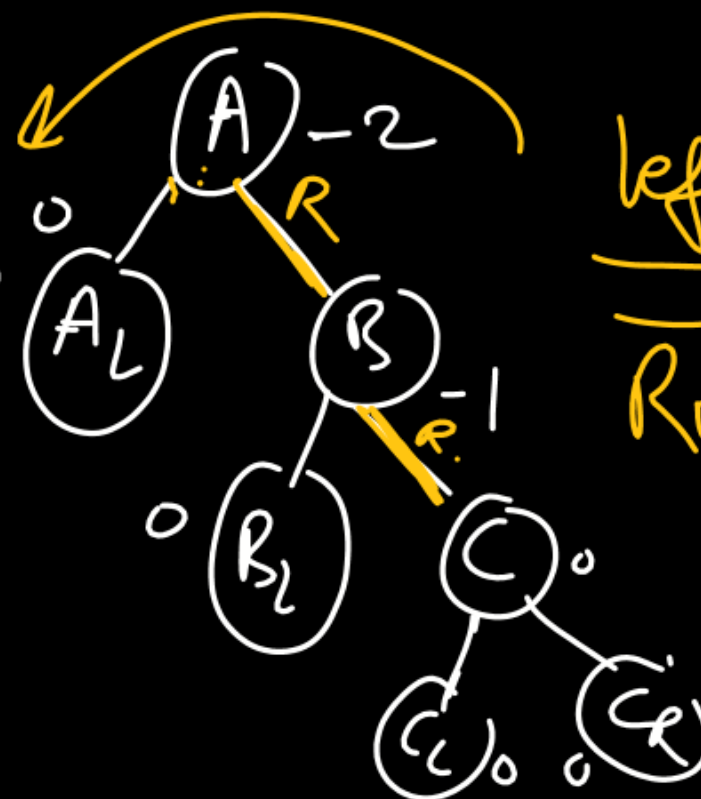
Right Rotate



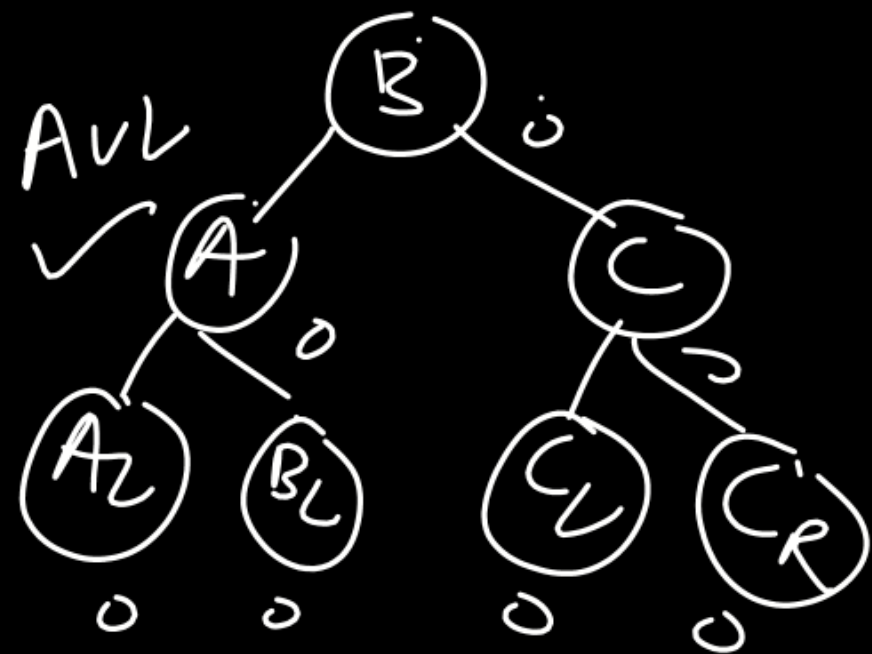
RR-Problem:-



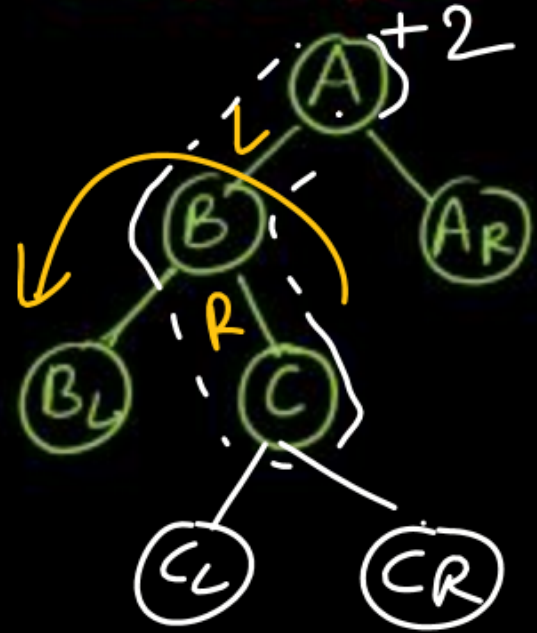
Insert
at C



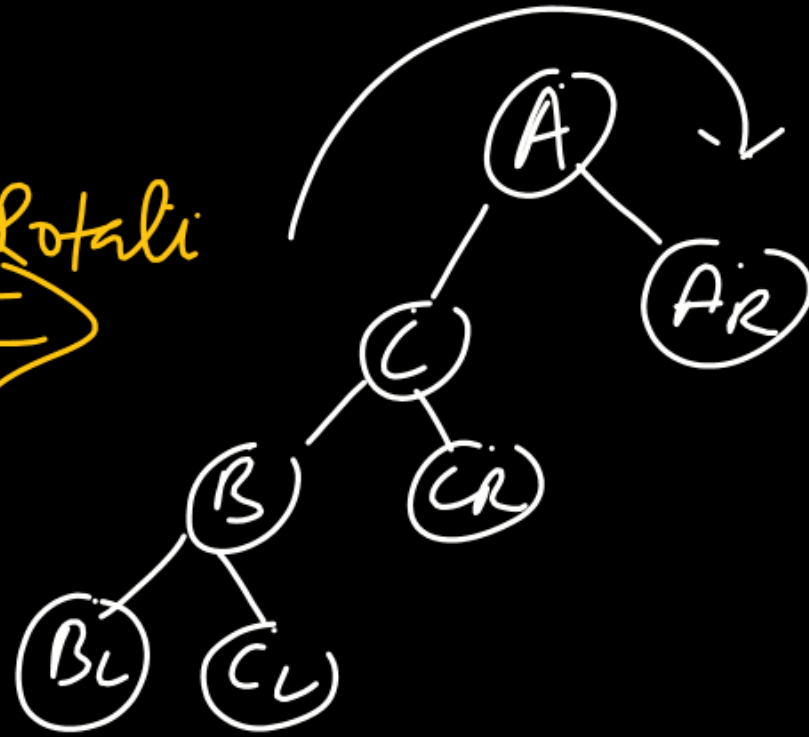
left
Rotate



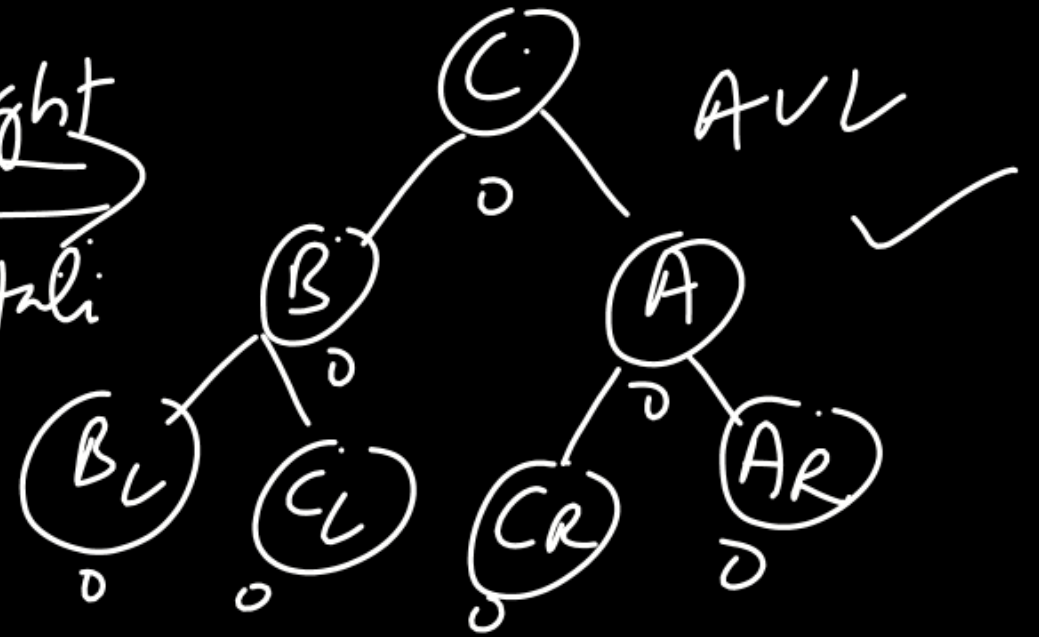
LR-Problem:-



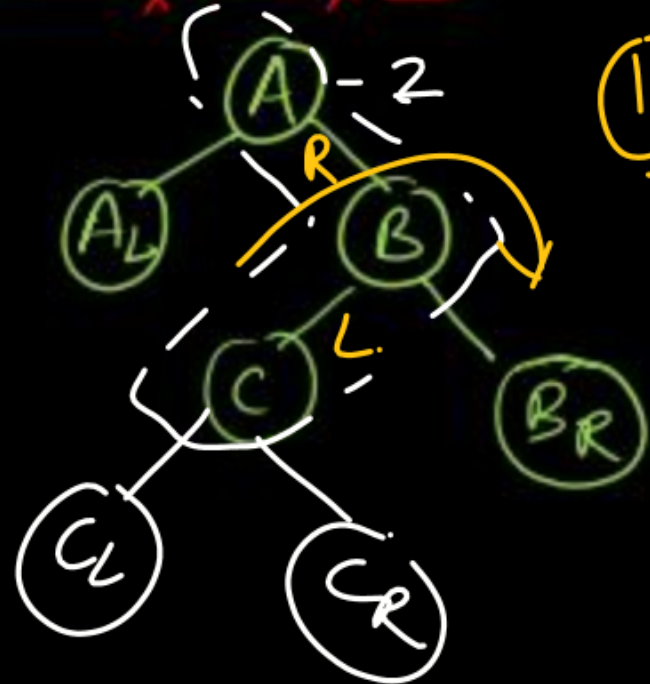
① Left Rotation



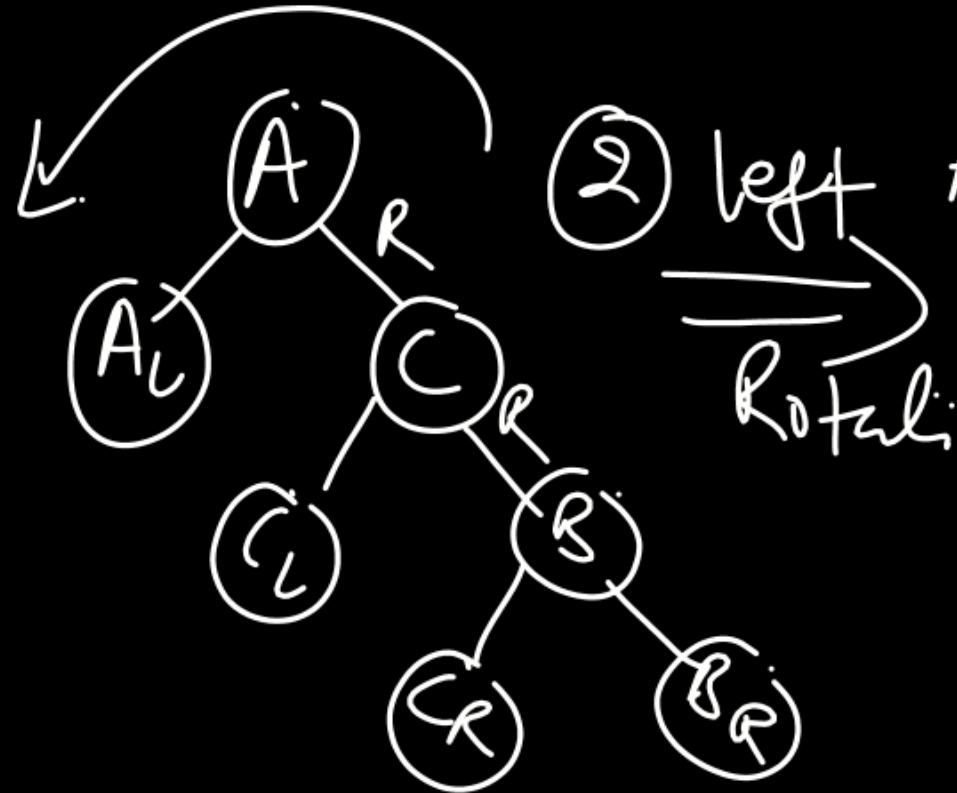
② Right Rotation



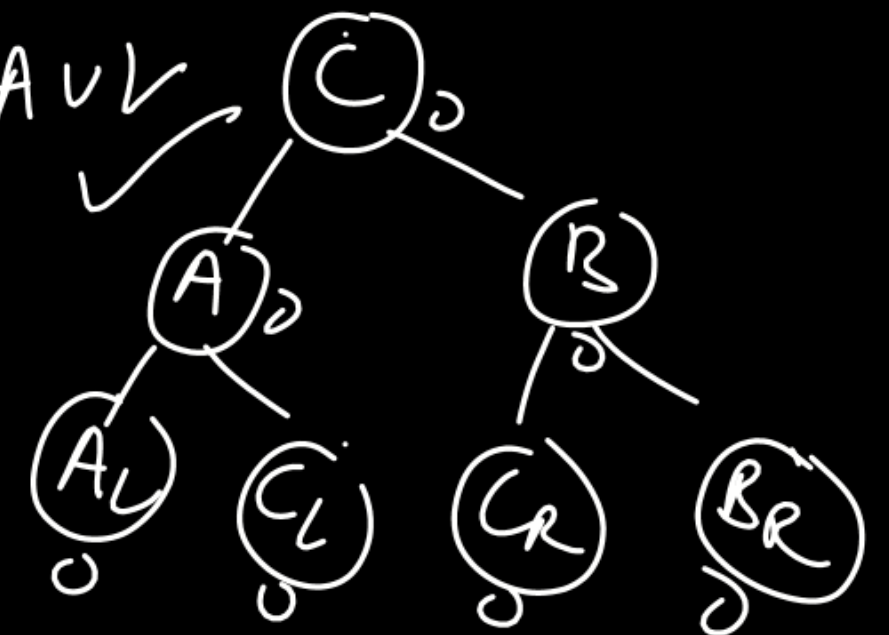
RL-Problem:-



① Right Rotation



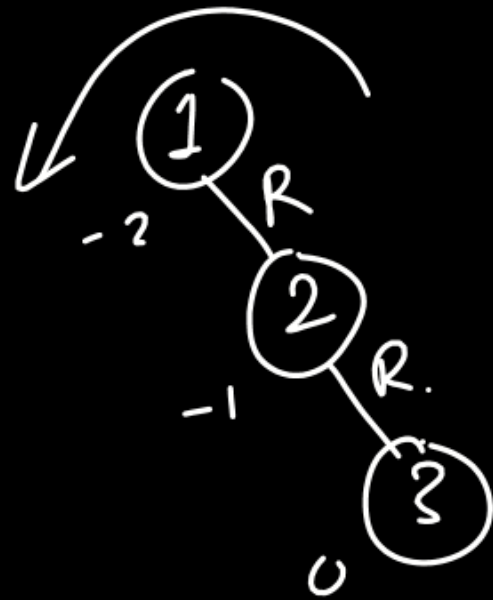
② Left Rotation



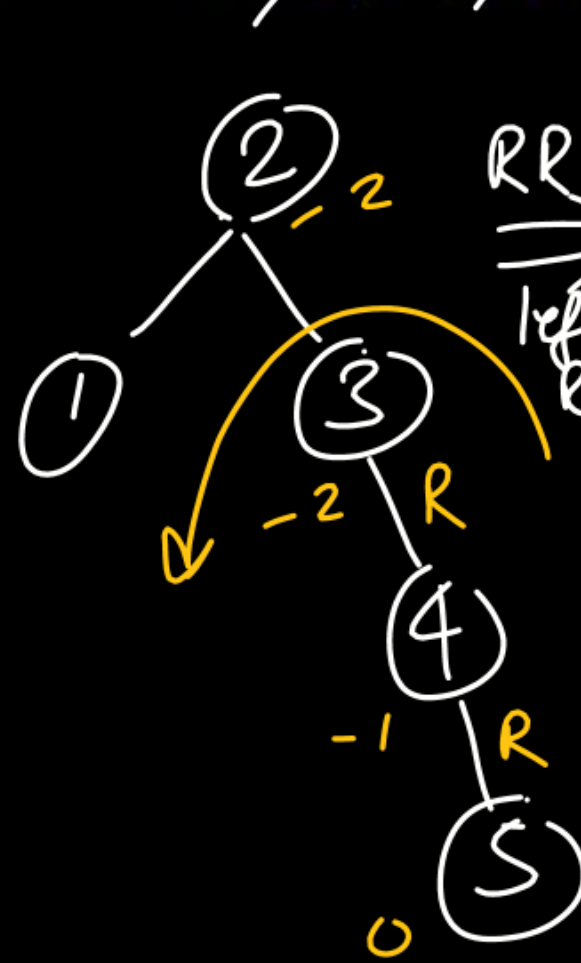
Construction of AVL-Trees :-

- ✓ 1) The Creation of AVL-tree is Same as BST by inserting one element at a time
- ✓ 2) When the new element is inserted the Balance factor of all its Ancestors will be updated
- ✓ 3) if any node is becoming imbalance then we need to apply the proper rotation technique & make it balanced.

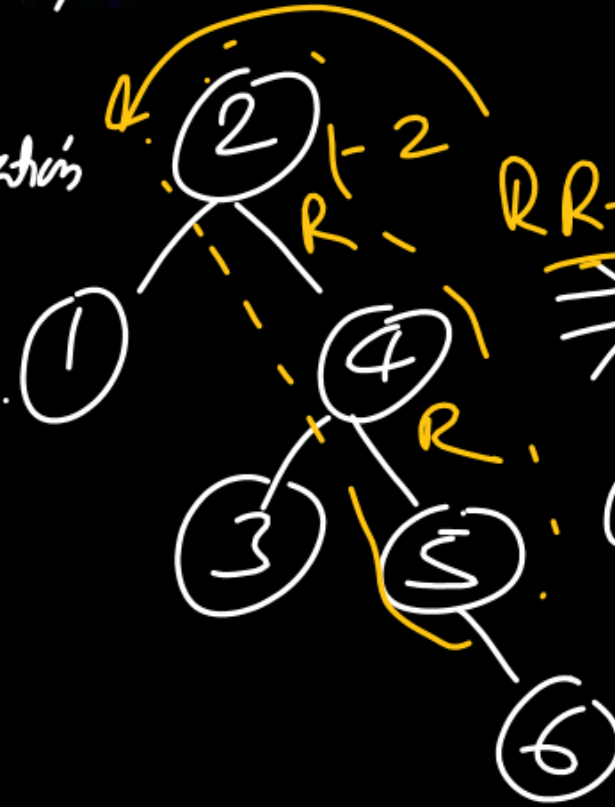
Ex:- Construct AVL-Tree for ~~1, 2, 3, 4, 5, 6, 7~~



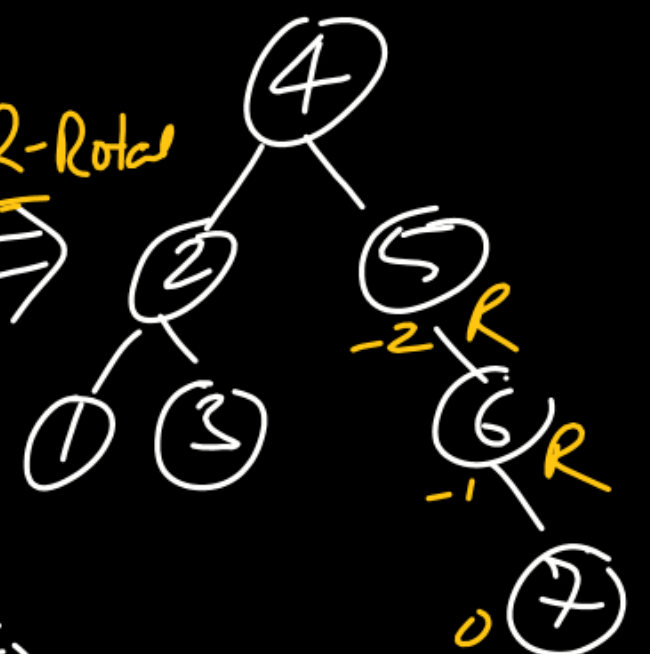
RR-Rotation
 $\Rightarrow$
 Left Rotation



RR Rotation
 $\Rightarrow$
 Left Rotation

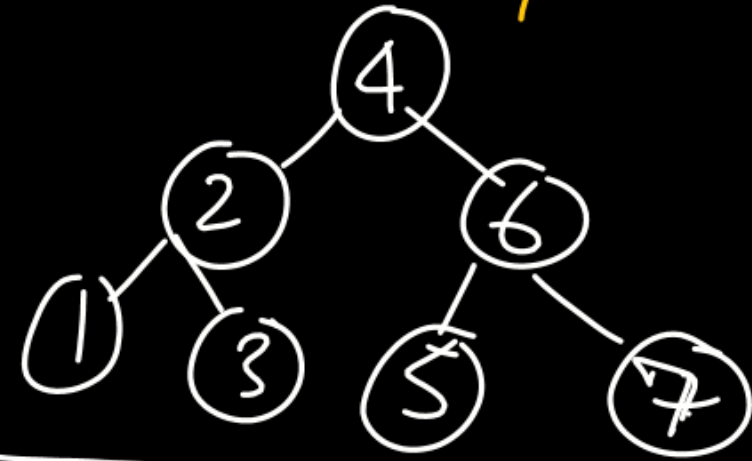


RR-Rotation



RR Rotation..

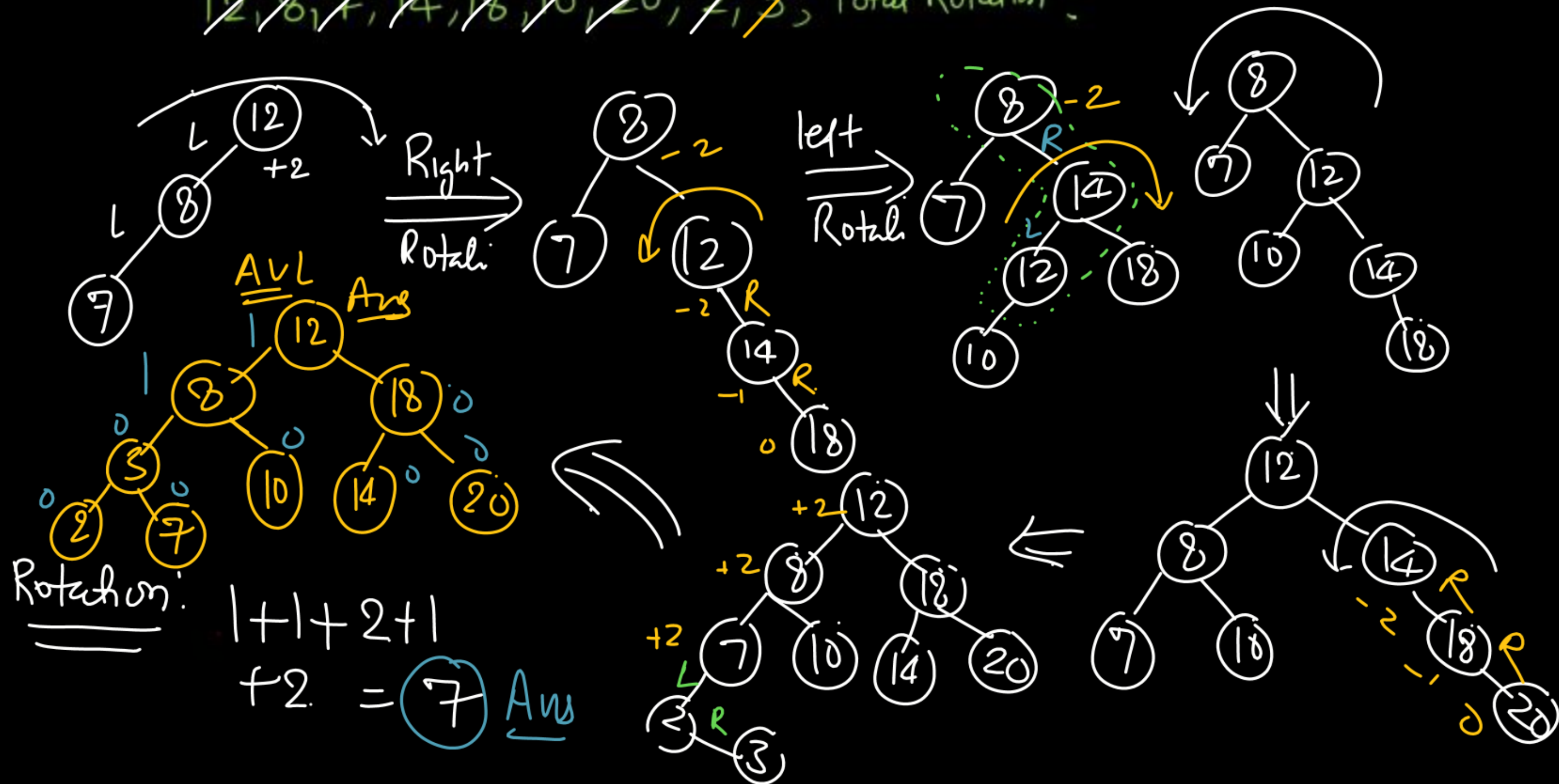
$O(\log n)$



AVL = BBST

Question:- Construct the AVL-tree for the given keys

12, 8, 7, 14, 18, 10, 20, 2, 3, Total Rotation?



① $\Rightarrow$ RR-Rotation: $\Rightarrow$ Single Rotation $\Rightarrow$ Left Rotate

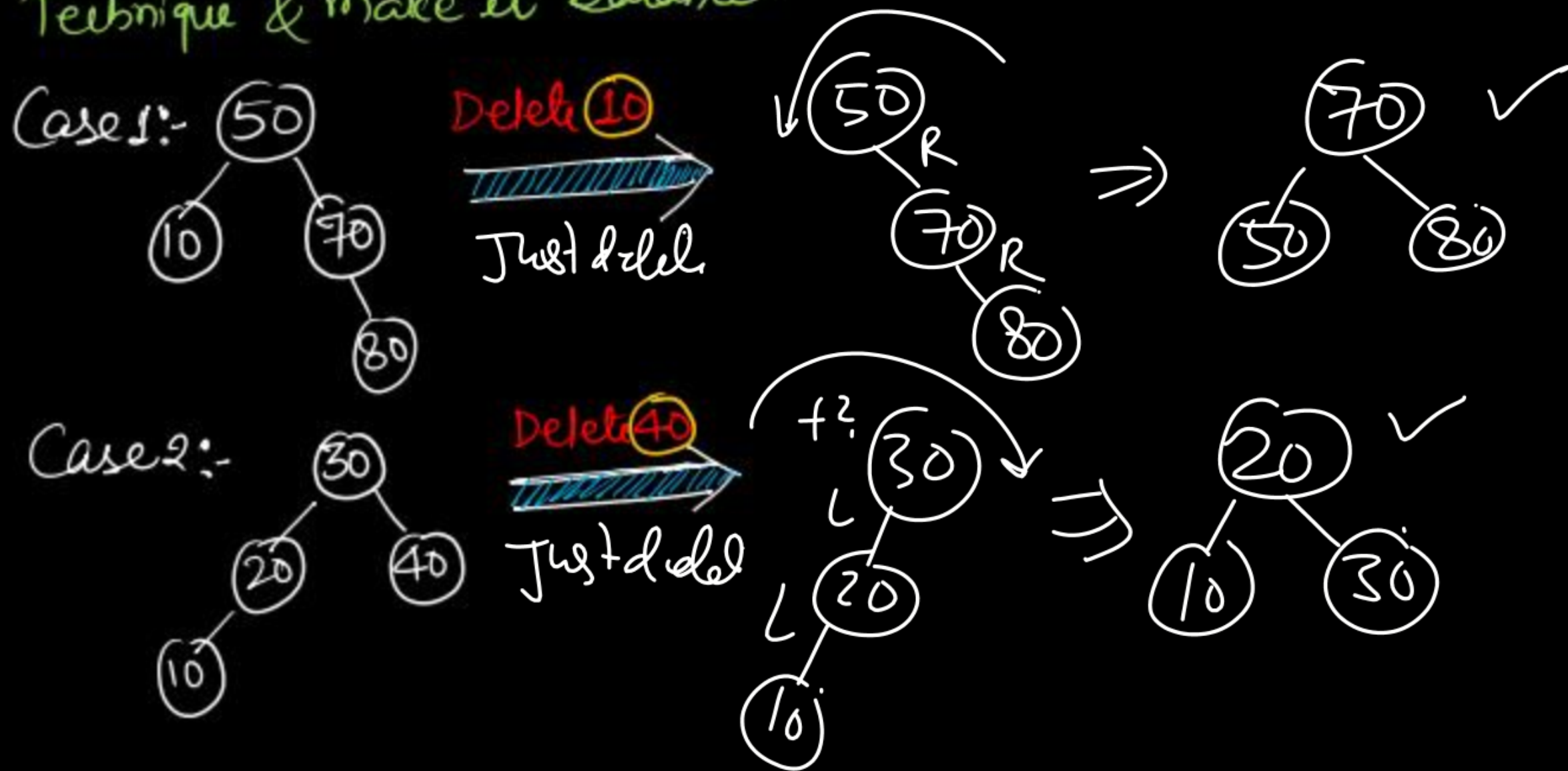
① $\Rightarrow$ LL-Rotation $\Rightarrow$ Single Rotation $\Rightarrow$ Right Rotate

② $\Rightarrow$ LR-Rotation $\Rightarrow$ Double Rotation $\Rightarrow$ 1) Left Rotate
2) Right Rotate

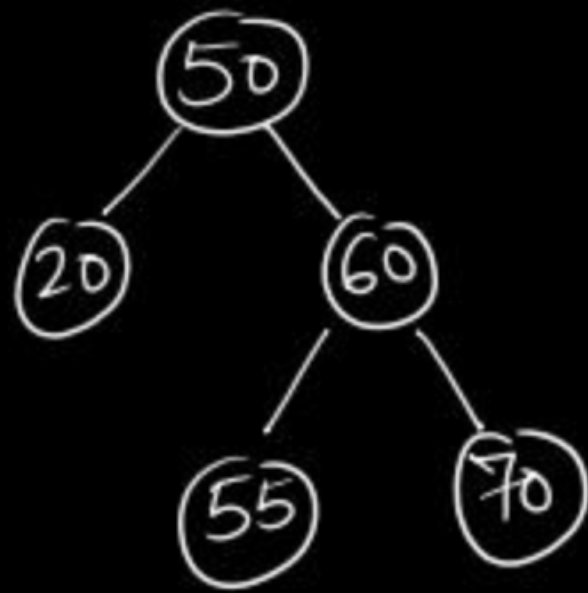
② $\Rightarrow$ RL-Rotation $\Rightarrow$ Double Rotation $\Rightarrow$ 1) Right Rotate
2) Left Rotate

Deletion in AVL-Tree:-

Deletion in the AVL-tree is same as the BST but after deletion check the Balance factor of every node if any node is becoming Imbalance then apply the Proper Rotation Technique & make it balance.



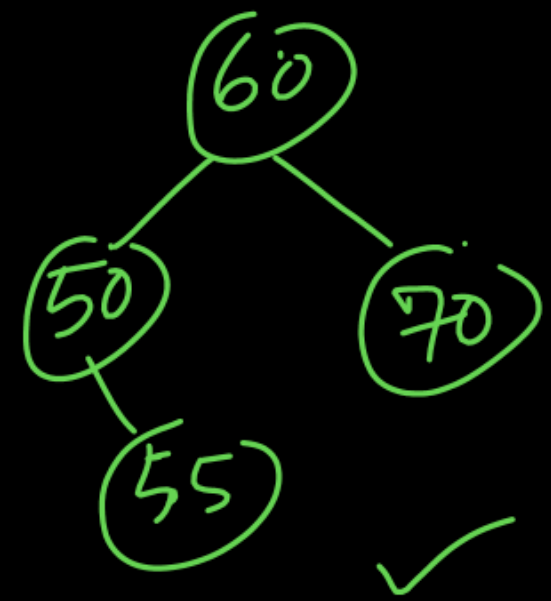
Case 3:-



Delete 20
Just delete

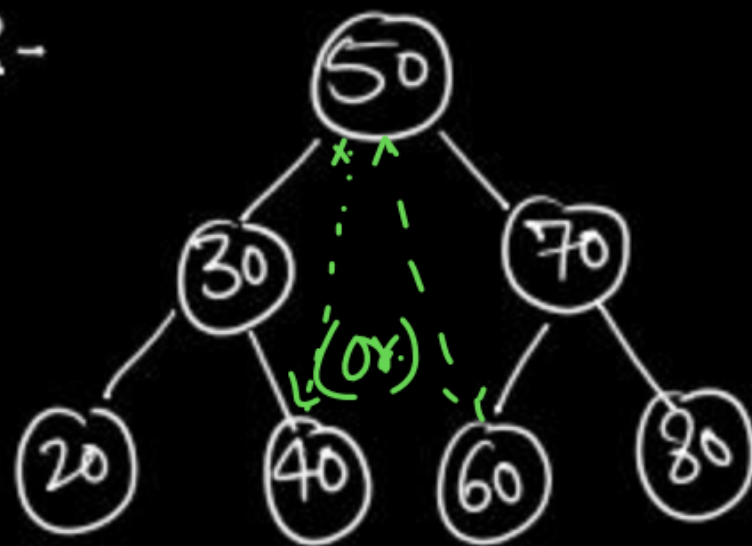


Left
Rotation

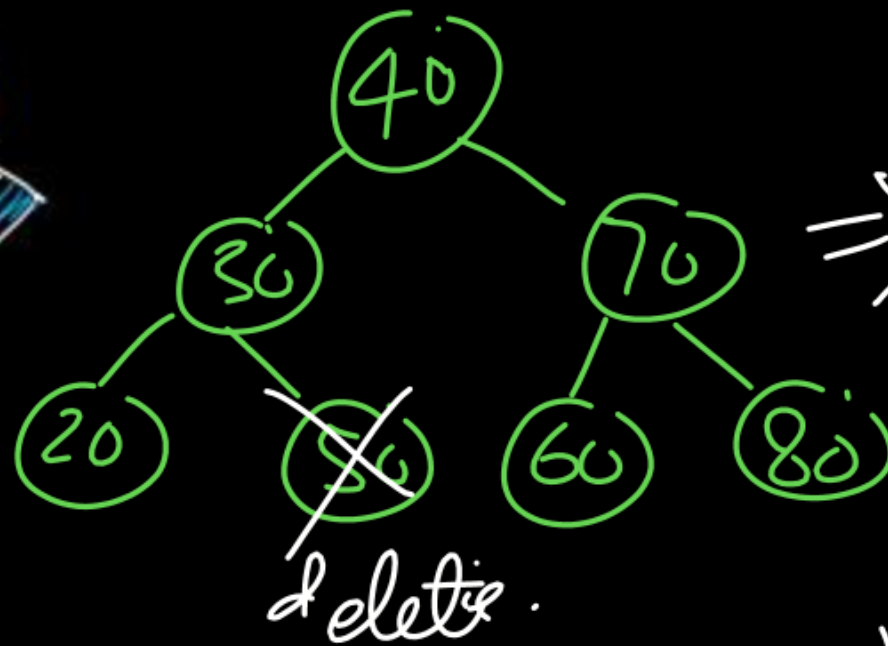


AVL

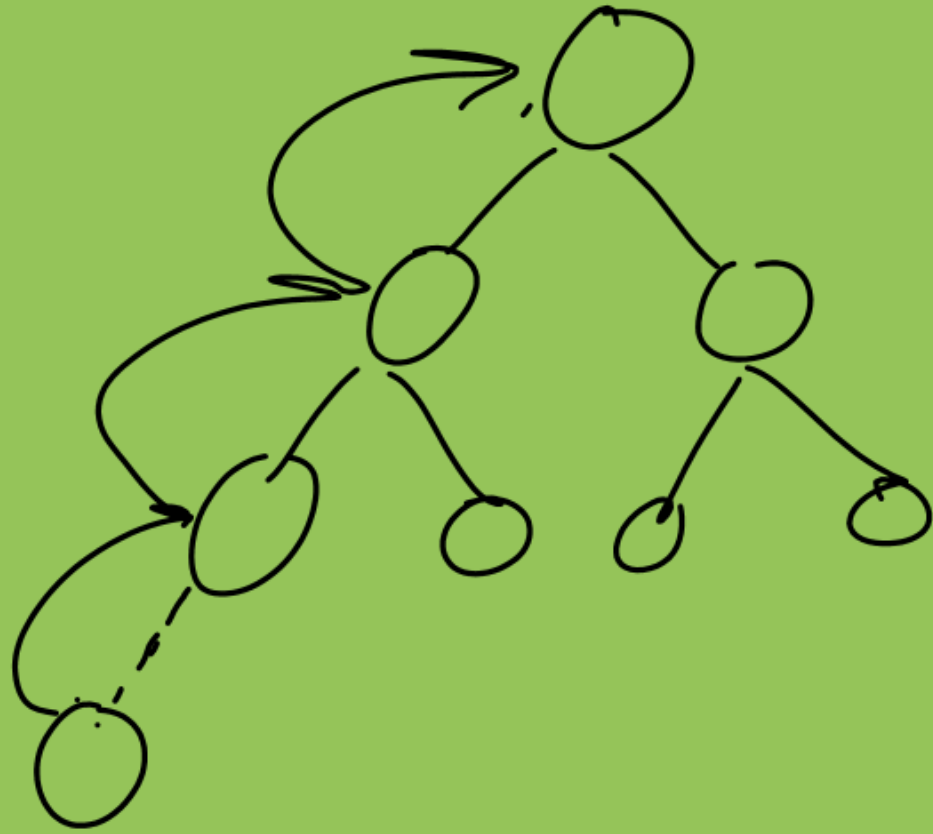
Case 4:-



Delete 50



AVL



Search $\Rightarrow O(\log_2 n)$

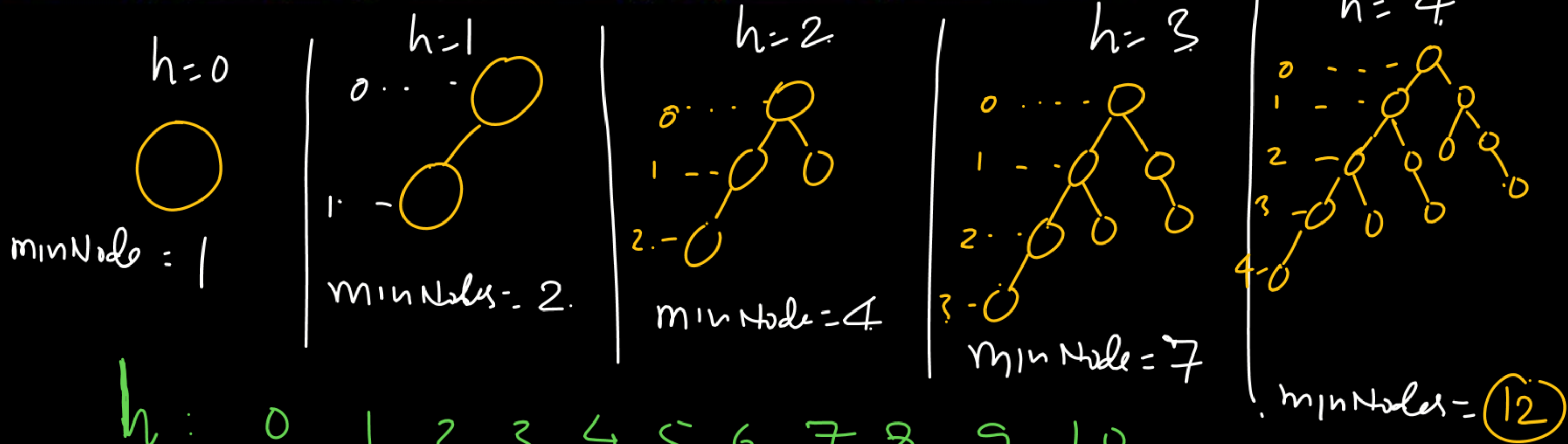
Link Add $\Rightarrow O(1)$

Backtrack $\Rightarrow O(\log_2 n)$

Insertable Rotation $\Rightarrow O(1)$

$$T_c = O(\log_2 n)$$

Maximum height and minimum No. of nodes in AVL-Tree:-



h :	0	1	2	3	4	5	6	7	8	9	10
n _{min} :	1	2	4	7	12	20	33	54	88	143	232

$$N(h) = N(h-1) + N(h-2) + 1$$

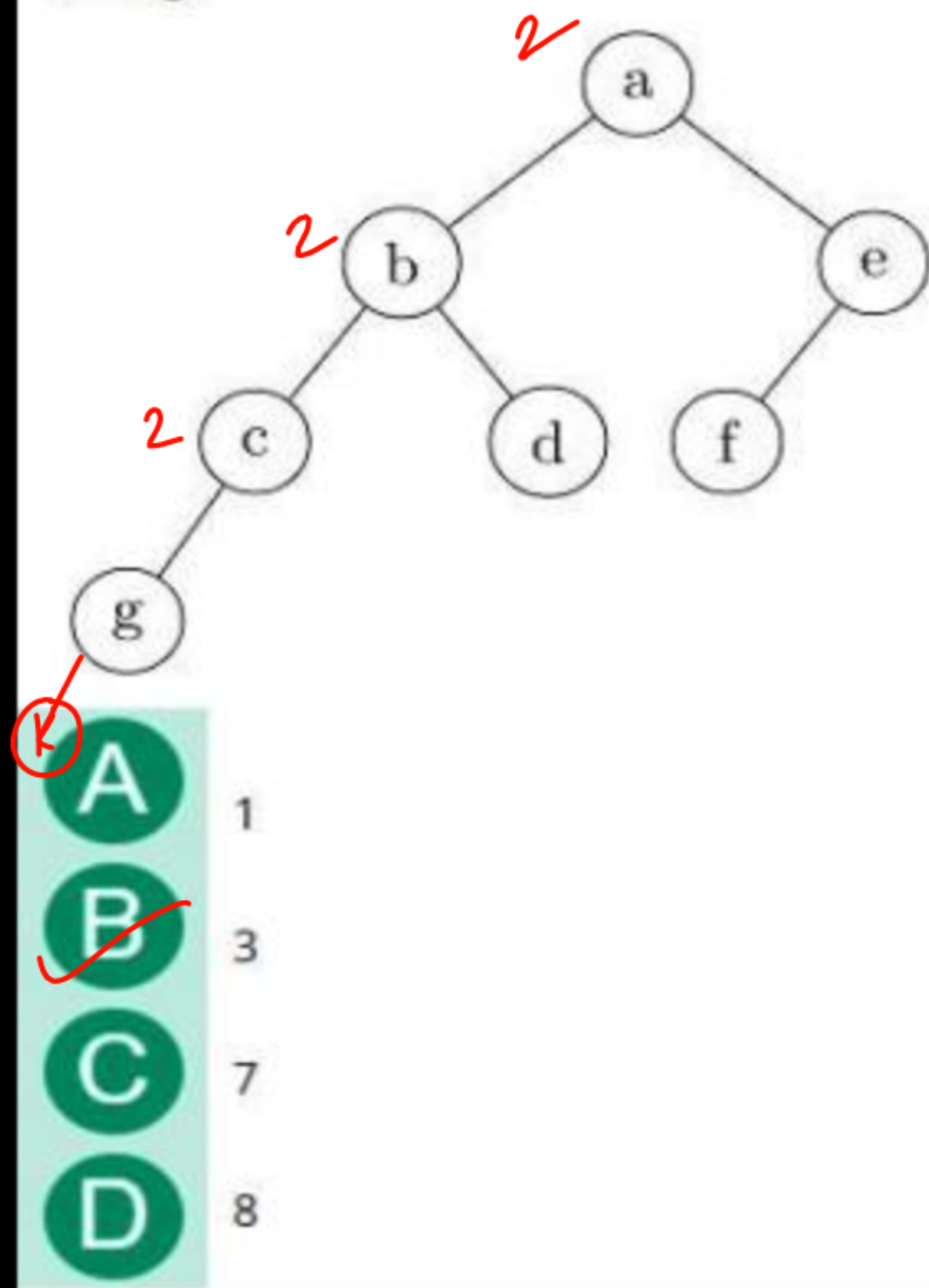
Properties of AVL Tree:-

- 1) maximum Possible No. of nodes in AVL-Tree of Height $h \Rightarrow 2^{h+1} - 1$
- 2) minimum No. of nodes in AVL-Tree of Height h is given by a Recurrence Relation
$$N(h) = N(h-1) + N(h-2) + 1$$

min

NOTE:- worst case search time Complexity for an element in AVL-Tree is $O(\log n)$

In the balanced binary tree in the below figure, how many nodes will become unbalanced when a node is inserted as a child of the node g?



The worst case running time to search for an element in a balanced binary search tree with $n2^n$ elements is

A $\Theta(n \log n)$

B $\Theta n 2^n$

☒ C $\Theta(n)$

D $\Theta(\log n)$

$\Downarrow$
 $O(\log_2 n)$

$$\begin{aligned}\log_2(n \cdot 2^n) &\Rightarrow \log_2 n + \log_2 2^n \\ &= \log_2 n + n \Rightarrow \underline{\underline{O(n)}}\end{aligned}$$

Binary Heap: CBT with Heap ordered property.

1) CBT

and

2) Heap ordered property.

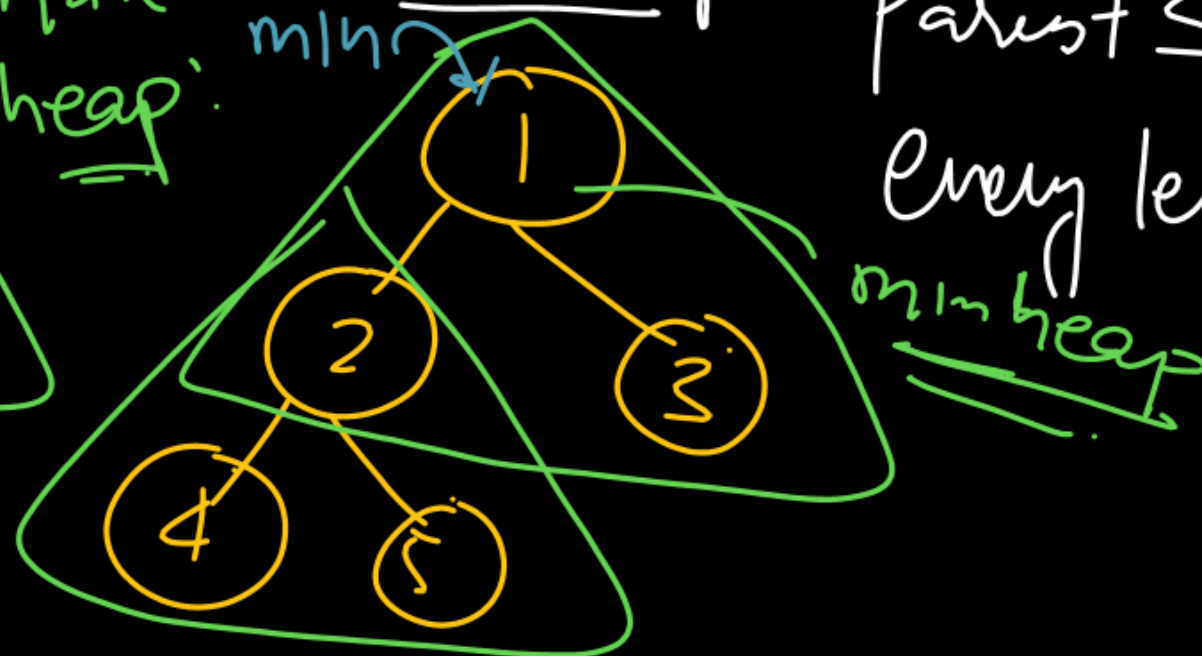
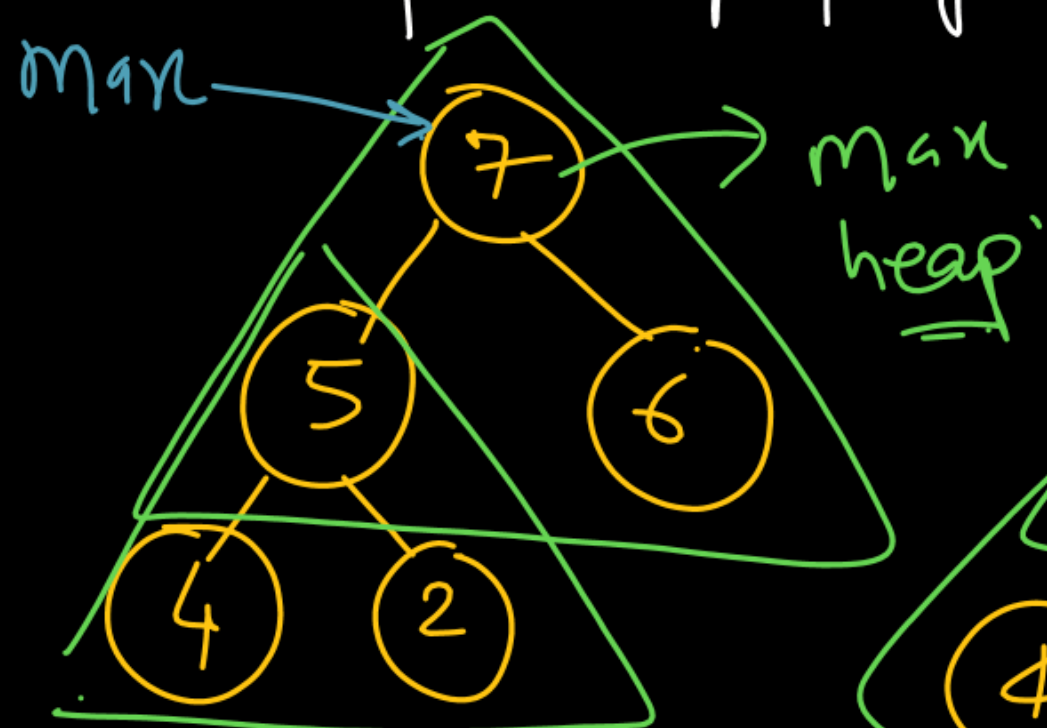
max heap:

(or)

min heap:

Parent $\geq$ left and Right Child
at every level of the tree.

Parent $\leq$ left and Right Child at
Every level of the tree.



Binary Heap:

A Binary Heap is a special type of binary tree that satisfies two main properties:

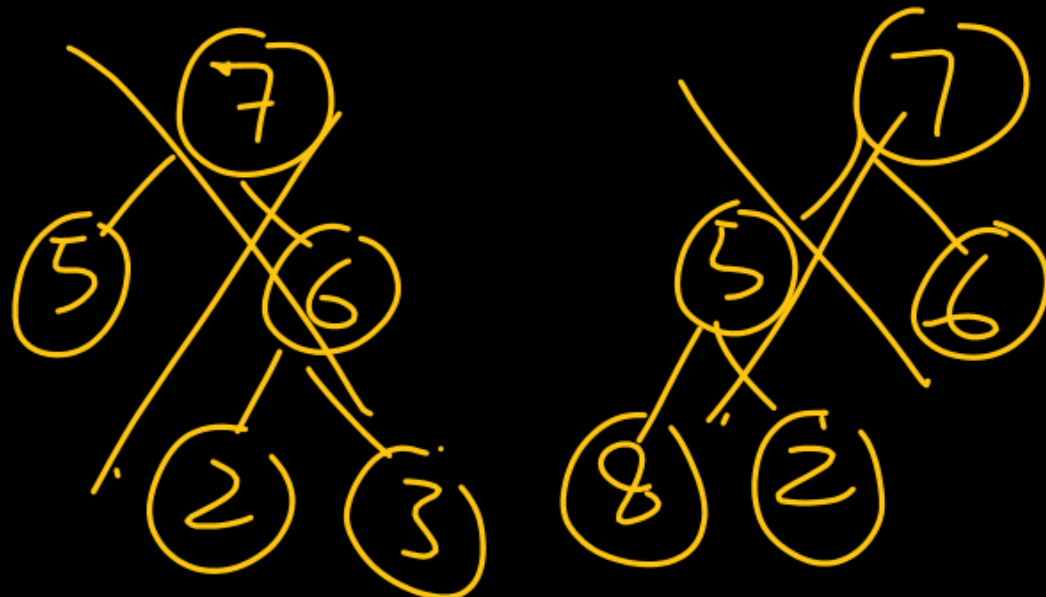
✓ 1. Heap Order Property

- Max Heap: The parent node is always greater than or equal to its children.
- (or) Min Heap: The parent node is always smaller than or equal to its children.

(and)

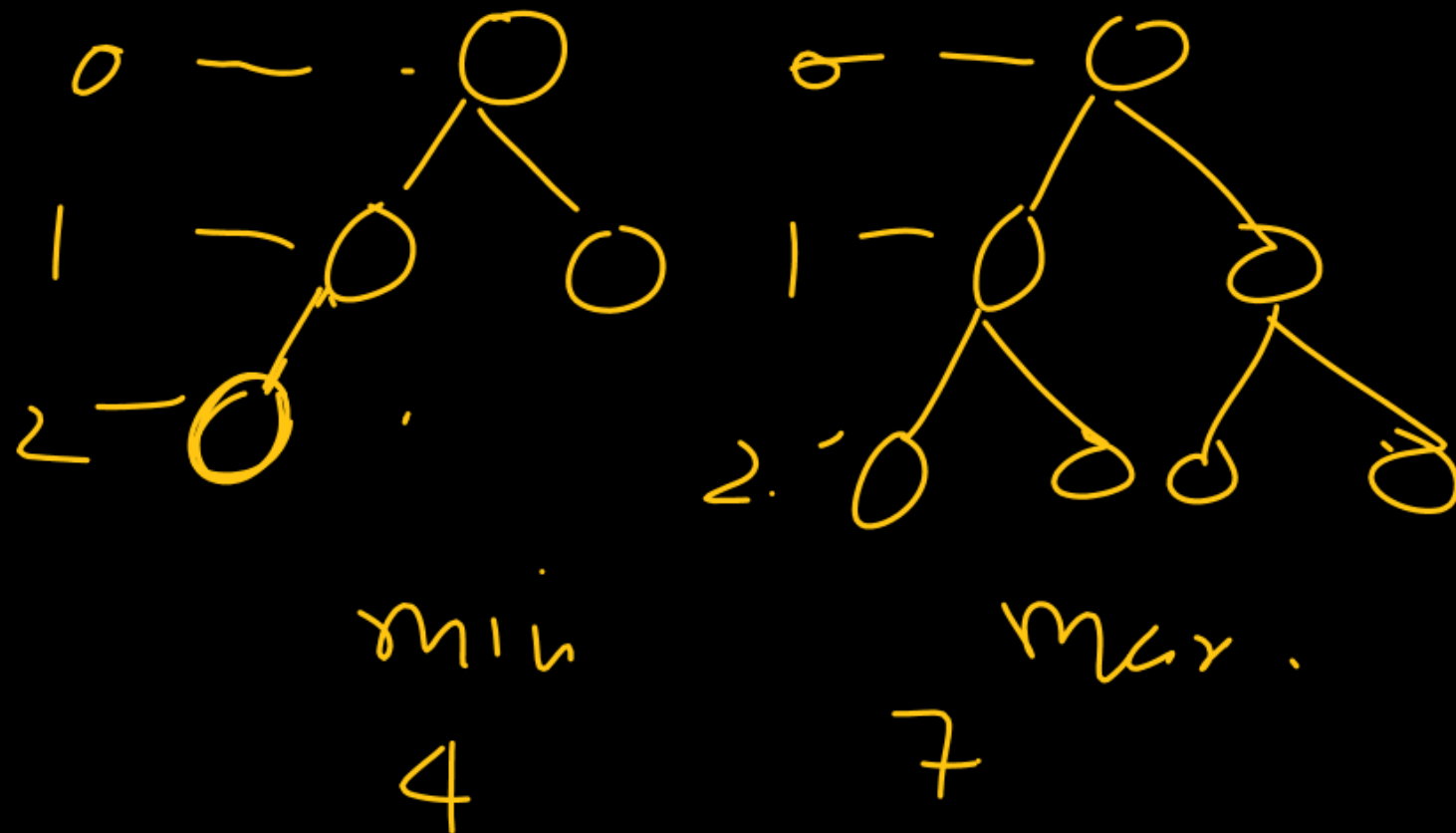
2. Complete Binary Tree Property

- The tree is completely filled at all levels, except possibly the last level, which is filled from left to right.



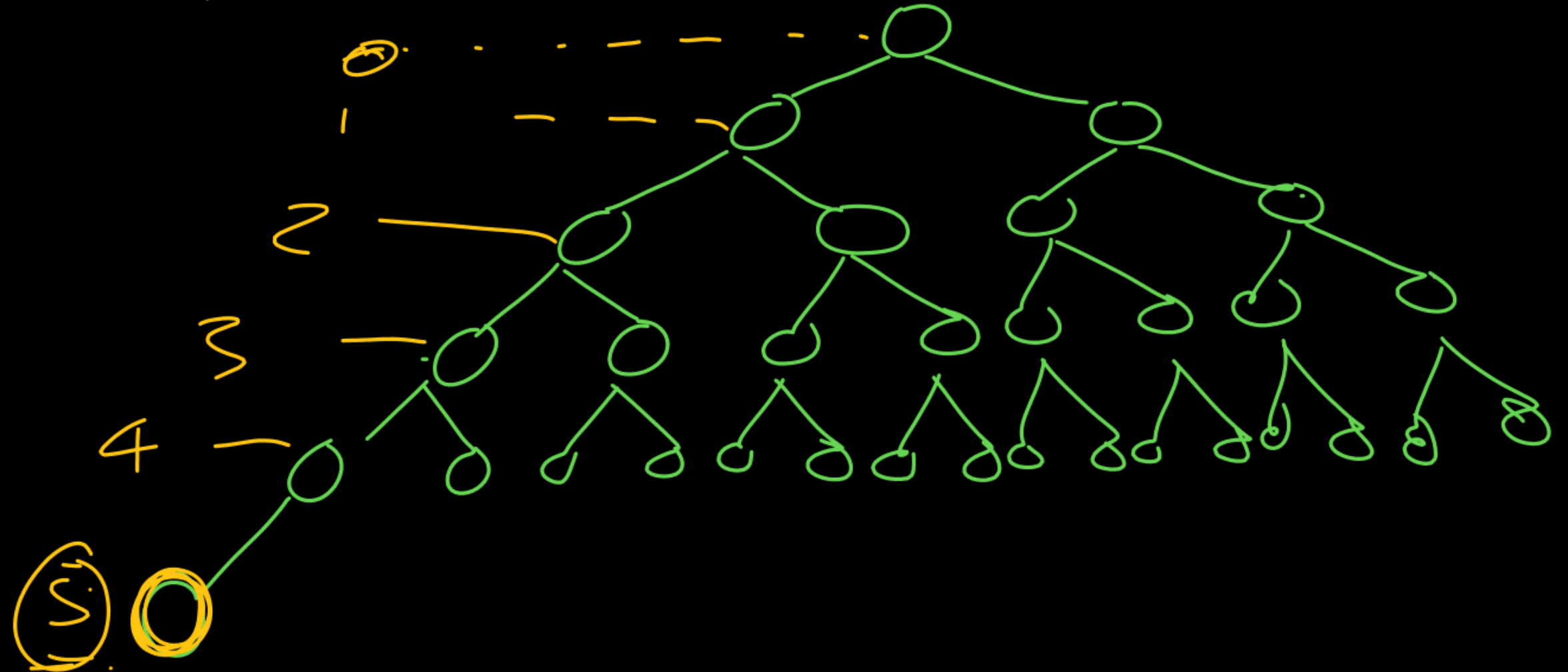
Min & max elements in heap on height(h):-

$$2^h \leq N \leq 2^{h+1} - 1$$



The height of any rooted tree is defined as the maximum number of edges in the path from the root node to any leaf node.

Suppose a Min-Heap T stores 32 keys. The height of T is 5.
(Answer in integer)



Representation of Binary Heap:-

Array:-

Parent(i)

- return $\left\lfloor \frac{i}{2} \right\rfloor$

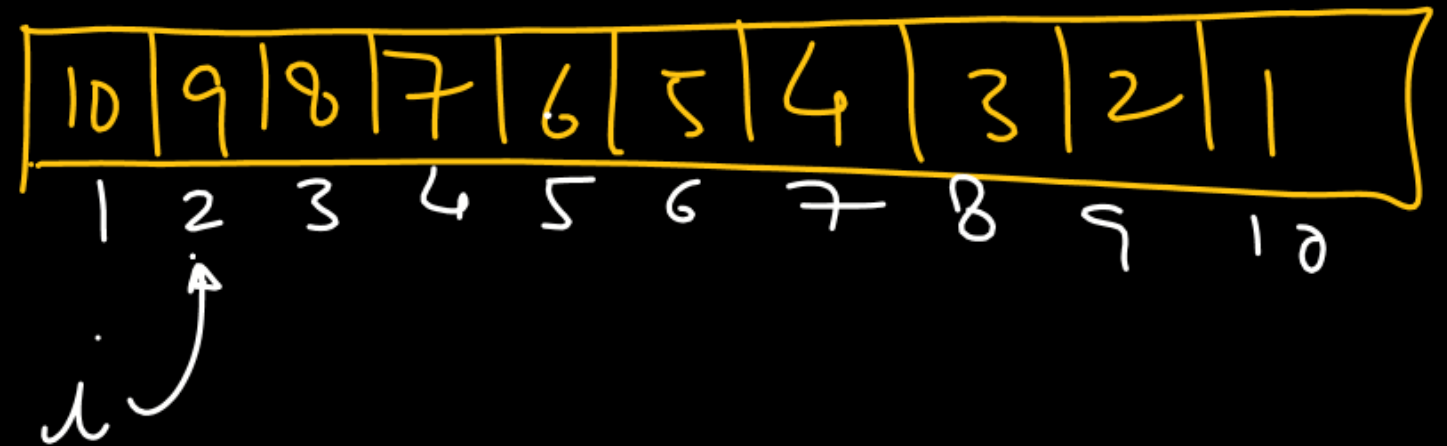
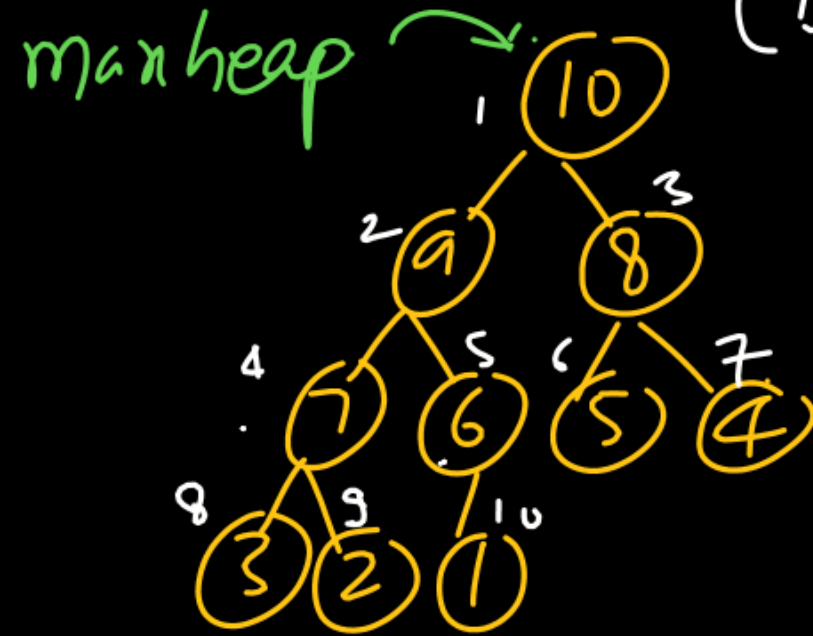
left(i)

- return $2i$

Right(i)

- return $2i+1$

Array (Complete Binary Tree.)
or
Linked List
(Binary Heap)





Shailendra Singh GATE CSE/DA

THANKYOU



GeeksforGeeks GATE